

硬體描述語言與邏輯合成實驗作業 - HW8

學號	姓名	檔名檢查	PDF 檔檢查
01153147	邱哲暘	(要自己檢查)	(要自己檢查)

* 主程式碼、測試程式碼皆需要截圖到作業上，波型圖需截圖

* 需用 pdf 檔繳交

* 請務必按照題目之設定撰寫程式

* 所有題目的 FSM 狀態皆須顯示文字

Q1. Traffic Light (FSM)

題目說明：

1. 請依下列的硬體架構圖(Fig. 1-1)去設計電路，
其中 cnt1, cnt2, sec1 及 sec2 為 4bits，led1 及 led2 為 3bits
2. 請依題目要求功能設計此題之 FSM 狀態(Fig. 1-2)，功能說明如下：

(所有狀態皆須顯示其對應之燈號) (Fig. 1-3)

IDLE: 該狀態為起始狀態，測試設備是否正常，需進行以下動作：

1. 將該 FSM 的狀態表示設置為"IDLE"
2. 將 led1 及 led2 的全部燈號開啟 (3'b111)並將其燈號的狀態表示為"N"
3. 將 sec1 及 sec2 的數值設置為 15

LED2_G: 此時 led1 為紅燈，sec1 從 8 秒進行倒數；此時 led2 為綠燈，sec2 從 6 秒進行倒數；

1. 將該 FSM 的狀態表示設置為"LED2_G"
2. 將其燈號的狀態表示為"R"，並顯示倒數的秒數 sec1
3. 將其燈號的狀態表示為"G"，並顯示倒數的秒數 sec2
4. 判斷綠燈是否倒數完成(倒數到 1)，將狀態跳轉到 LED2_Y

LED2_Y: 此時 led1 為紅燈，sec1 繼續進行倒數；此時 led2 為黃燈，sec2 從 2 秒進行倒數；

1. 將該 FSM 的狀態表示設置為"LED2_Y"
2. 將其燈號的狀態表示為"R"，並顯示倒數的秒數 sec1
3. 將其燈號的狀態表示為"Y"，並顯示倒數的秒數 sec2
4. 判斷黃燈是否倒數完成(倒數到 1)，將狀態跳轉到 LED1_G

LED1_G: 此時 led1 為綠燈，sec1 從 11 秒進行倒數；此時 led2 為紅燈，sec2 從 13 秒進行倒數；

1. 將該 FSM 的狀態表示設置為"LED1_G"
2. 將其燈號的狀態表示為"G"，並顯示倒數的秒數 sec1
3. 將其燈號的狀態表示為"R"，並顯示倒數的秒數 sec2
4. 判斷綠燈是否倒數完成(倒數到 1)，將狀態跳轉到 LED1_Y

LED1_Y: 此時 led1 為黃燈，sec1 從 2 秒進行倒數；此時 led2 為紅燈，sec2 繼續進行倒數；

1. 將該 FSM 的狀態表示設置為"LED1_Y"
2. 將其燈號的狀態表示為"Y"，並顯示倒數的秒數 sec1
3. 將其燈號的狀態表示為"R"，並顯示倒數的秒數 sec2
4. 判斷黃燈是否倒數完成(倒數到 1)，將狀態跳轉到 LED2_G

3. 依據 testbench 的程式片段撰寫測試程式 (Fig. 1-4)，波型圖如下 (Fig. 1-5):

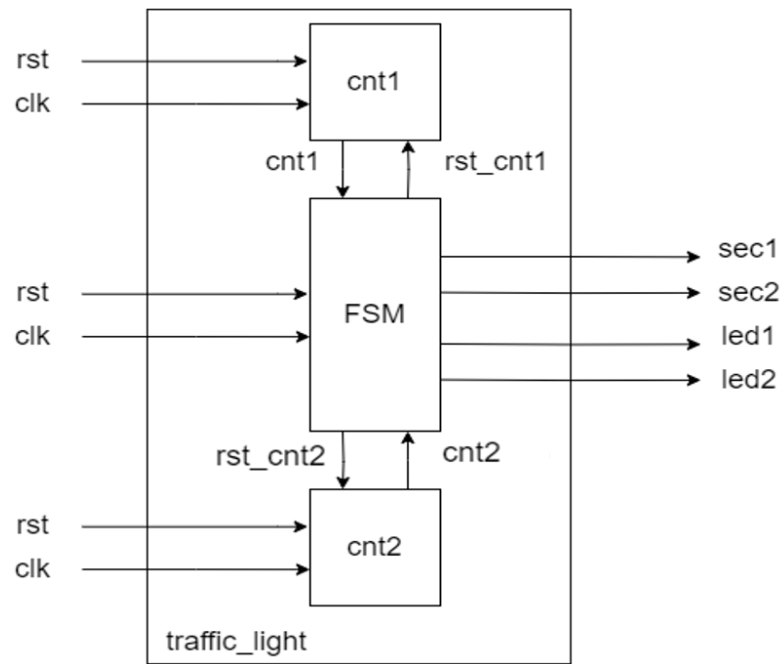


Fig. 1-1 硬體架構圖

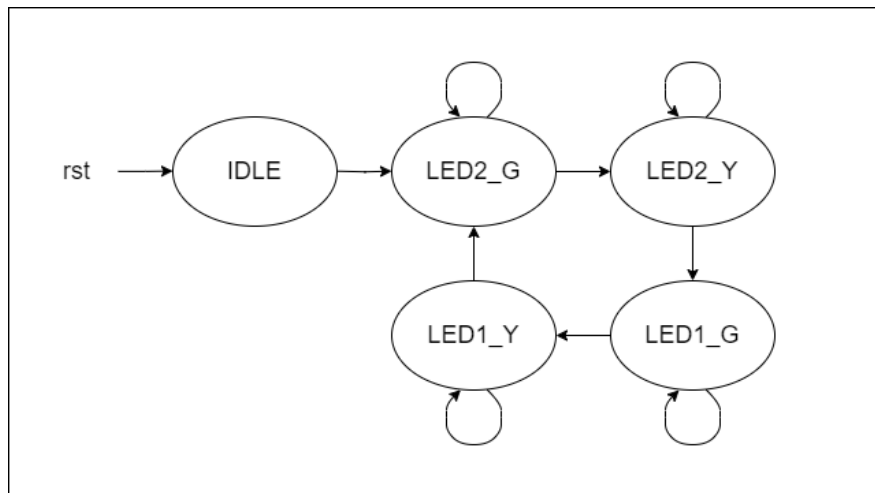


Fig. 1-2 狀態圖

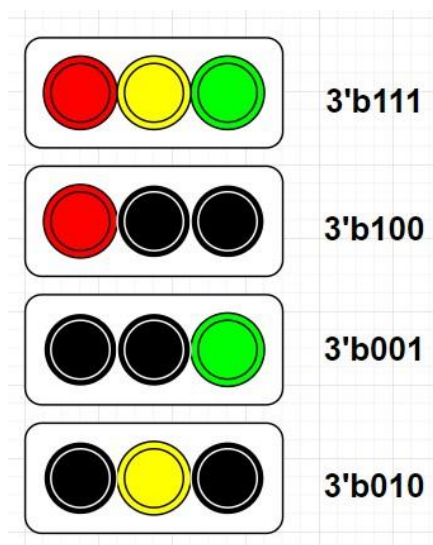


Fig. 1-3 燈號顯示圖

```

always #5 clk = ~clk;
initial begin
    rst = 1'b1; clk = 1'b0;
    #10    rst = 1'b0;
    #450   $stop;
end

```

Fig. 1-4 testbench 的程式片段



Fig. 1-5 波型圖

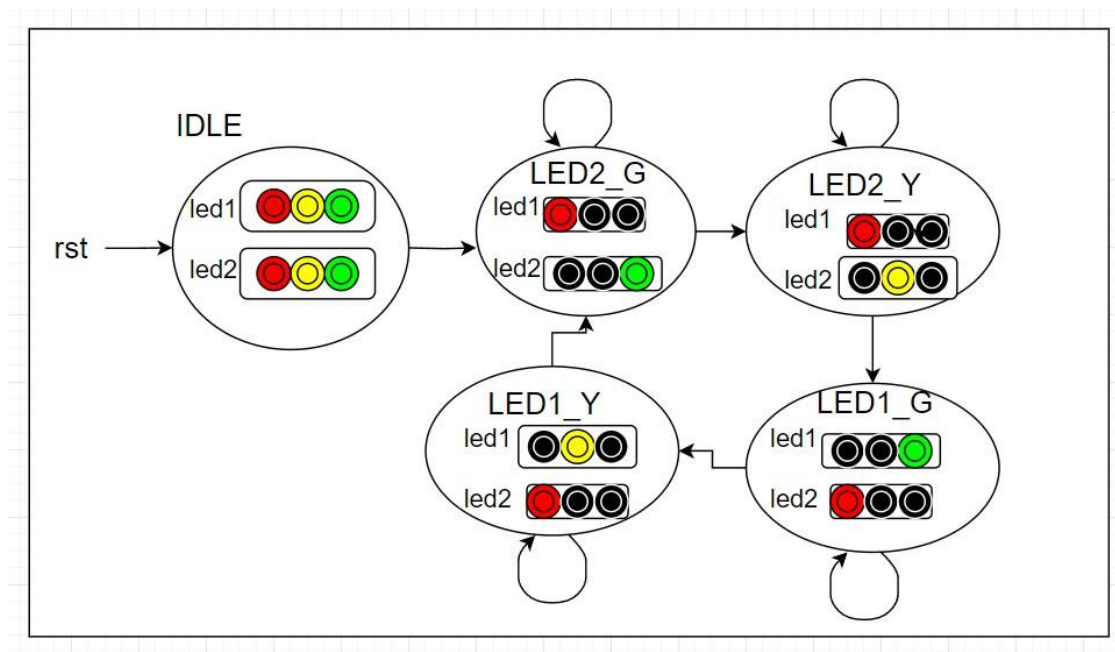


Fig. 1-6 FSM 及其對應燈號顯示圖

traffic_light.v

主程式碼截圖：

//放主程式碼截圖

```

1  module traffic_light(
2      input clk,
3      input rst,
4      output reg [3:0] sec1,
5      output reg [3:0] sec2,
6      output reg [2:0] led1,
7      output reg [2:0] led2
8  );
9
10     reg [3:0] cnt1,cnt2;
11     reg cnt1_rst,cnt2_rst;
12     reg [3:0] ps,ns;
13     reg [(8*6-1):0] state;
14     reg [(8*1-1):0] led1_state,led2_state;
15
16     parameter IDLE      = 4'd0,
17                 LED2_G   = 4'd1,
18                 LED2_Y   = 4'd2,
19                 LED1_G   = 4'd3,
20                 LED1_Y   = 4'd4;
21
22     parameter N   = 3'b111,
23                 R   = 3'b100,
24                 Y   = 3'b010,
25                 G   = 3'd001;
26
27     always @(*)
28     begin
29         case (ps)
30             4'd0: state = "IDLE";
31             4'd1: state = "LED2_G";
32             4'd2: state = "LED2_Y";
33             4'd3: state = "LED1_G";
34             4'd4: state = "LED1_Y";
35         endcase
36     end
37
38     always @(*)
39     begin
40         case (led1)
41             3'b111: led1_state = "N";
42             3'b100: led1_state = "R";
43             3'b010: led1_state = "Y";
44             3'b001: led1_state = "G";

```

```

44         3'b001: led1_state = "G";
45     endcase
46     case (led2)
47         3'b111: led2_state = "N";
48         3'b100: led2_state = "R";
49         3'b010: led2_state = "Y";
50         3'b001: led2_state = "G";
51     endcase
52 end
53
54 always @(posedge clk)
55 begin
56     if (rst)
57     begin
58         state <= "IDLE";
59         ps <= IDLE;
60     end
61     else
62         ps <= ns;
63     end
64
65 always @(posedge clk)
66 begin
67     if (rst || cnt1_rst)
68     begin
69         cnt1 <= 4'd0;
70         cnt1_rst <= 1'b1;
71     end
72     else
73         cnt1 <= cnt1 + 1'b1;
74     end
75
76 always @(posedge clk)
77 begin
78     if (rst || cnt2_rst)
79     begin
80         cnt2 = 4'd0;
81         cnt2_rst = 1'b1;
82     end
83     else
84         cnt2 <= cnt2 + 1'b1;
85     end
86
87 always @(*)

```

```

88 begin
89     case (ps)
90     IDLE:
91     begin
92         led1 = 3'b111;
93         led2 = 3'b111;
94         sec1 = 4'd15 - cnt1;
95         sec2 = 4'd15 - cnt2;
96         ns   = 4'd1;
97     end
98 LED2_G:
99     begin
100         cnt1_rst = 1'b0;
101         cnt2_rst = 1'b0;
102         led1 = 3'b100;
103         led2 = 3'b001;
104         sec1 = 4'd8 - cnt1;
105         sec2 = 4'd6 - cnt2;
106         if (sec2 == 4'd1)
107         begin
108             cnt2_rst = 1'b1;
109             ns = LED2_Y;
110         end
111     end
112 LED2_Y:
113     begin
114         cnt2_rst = 1'b0;
115         led1 = 3'b100;
116         led2 = 3'b010;
117         sec1 = 4'd8 - cnt1;
118         sec2 = 4'd2 - cnt2;
119         if (sec2 == 4'd1)
120         begin
121             cnt1_rst = 1'b1;
122             cnt2_rst = 1'b1;
123             ns = LED1_G;
124         end
125     end
126 LED1_G:
127     begin
128         cnt1_rst = 1'b0;
129         cnt2_rst = 1'b0;
130         led1 = 3'b001;
131         led2 = 3'b100;

```

```

132     sec1 = 4'd11 - cnt1;
133     sec2 = 4'd13 - cnt2;
134     if (sec1 == 4'd1)
135     begin
136         cnt1_rst = 1'b1;
137         ns = LED1_Y;
138     end
139 end
140 LED1_Y:
141 begin
142     cnt1_rst = 1'b0;
143     led1 = 3'b010;
144     led2 = 3'b100;
145     sec1 = 4'd2 - cnt1;
146     sec2 = 4'd13 - cnt2;
147     if (sec1 == 4'd1)
148     begin
149         cnt1_rst = 1'b1;
150         cnt2_rst = 1'b1;
151         ns = LED2_G;
152     end
153 end
154 endcase
155 end
156 endmodule

```

traffic_light_test.v

測試程式碼截圖：

//放測試程式碼截圖

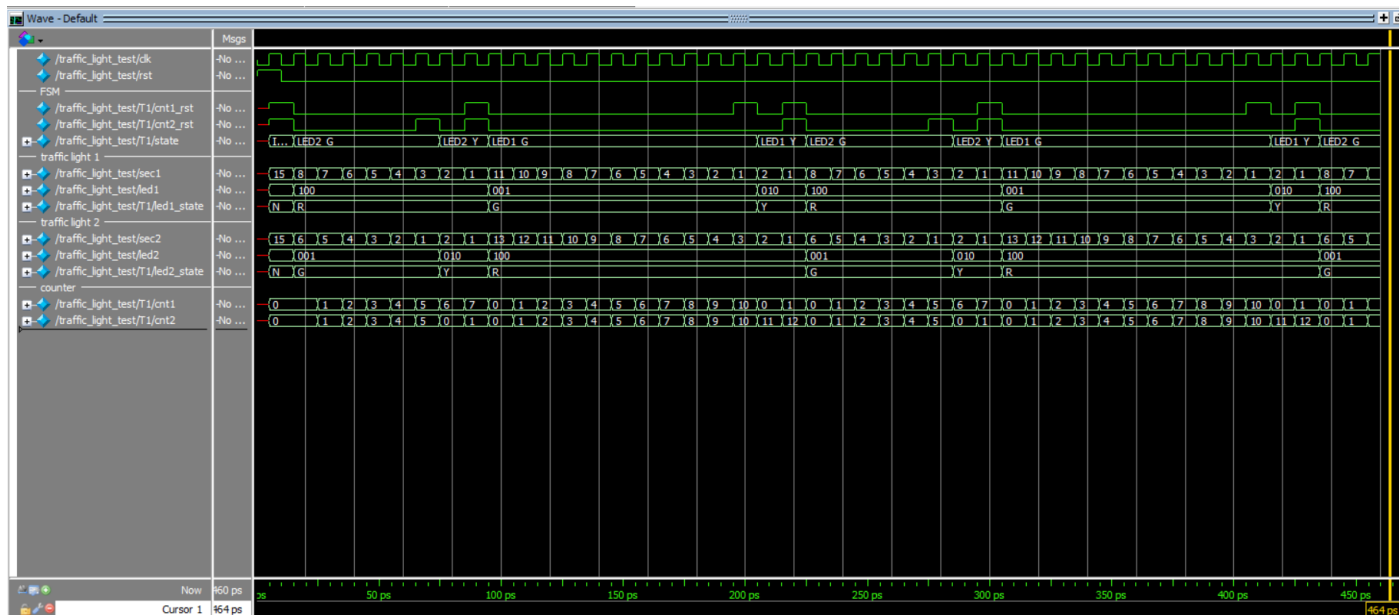
```

158 module traffic_light_test;
159 reg clk;
160 reg rst;
161 wire [3:0] sec1;
162 wire [3:0] sec2;
163 wire [2:0] led1;
164 wire [2:0] led2;
165
166 traffic_light T1(
167     .clk(clk),
168     .rst(rst),
169     .sec1(sec1),
170     .sec2(sec2),
171     .led1(led1),
172     .led2(led2)
173 );
174
175 always #5 clk = ~clk;
176 initial begin
177     rst = 1'b1; clk = 1'b0;
178     #10 rst = 1'b0;
179     #450 $stop;
180 end
181 endmodule

```

波型圖截圖：

//放波型圖截圖



Q2. Parking Space (FSM)

1. 請依下列的硬體架構圖(Fig. 2-1)去設計電路，計算 car 的停留時長，其中 **cnt** 及 **min** 為 16bits
2. car 為 1 時表示為車位上有車輛停留，0 時則無
3. 當 car 訊號由 0 變 1 時，將 enc_pos 設置為 1，其他時候則為 0 (使用 positive edge detector)
4. 當 car 訊號由 1 變 0 時，將 enc_neg 設置為 1，其他時候則為 0 (使用 negative edge detector)
5. 請依題目要求功能設計此題之 FSM 狀態(Fig. 2-2)，其功能說明如下：

IDLE: 該狀態為起始狀態，直接跳轉到 **WAIT_POS**

WAIT_POS: 等待 enc_pos，達成條件時將 cnt 進行 reset，狀態跳轉到 **WAIT_NEG**

WAIT_NEG: 等待 enc_neg，達成條件時將 dff 進行 load，狀態跳轉到 **WAIT_POS**

6. 依據 testbench 的程式片段撰寫測試程式 (Fig. 2-3)，波型圖如下 (Fig. 2-4)

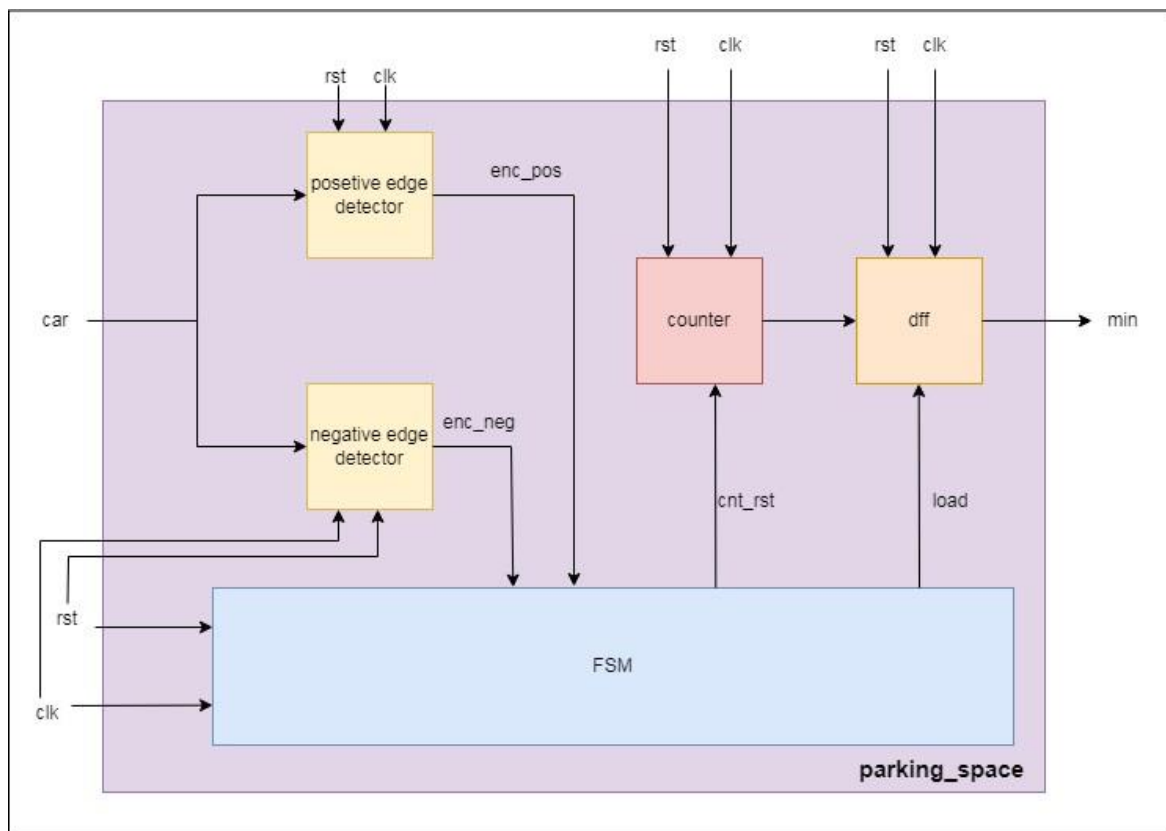


Fig. 2-1 硬體架構圖

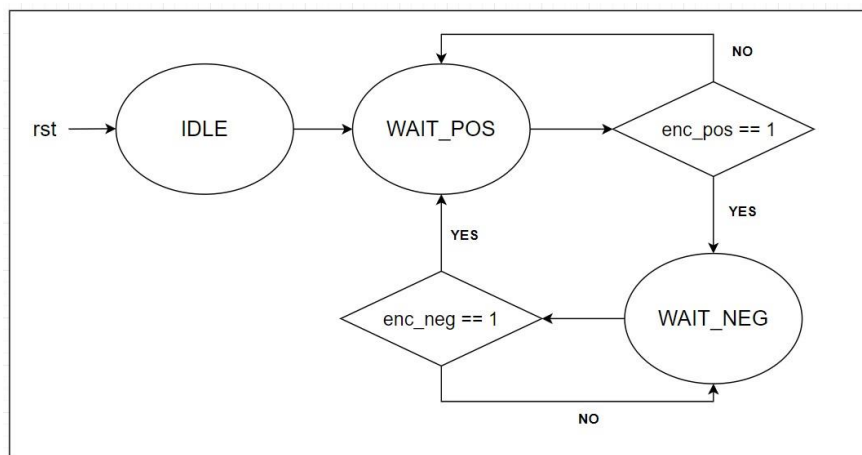


Fig. 2-2 狀態圖

```

always #5    clk = ~clk;
initial begin
    clk = 1'b0; rst = 1'b1; car = 1'b0;
    #10      rst = 1'b0;
    #50      car = 1'b1;
    #50      car = 1'b0;
    #100     car = 1'b1;
    #150     car = 1'b0;
    #350     car = 1'b1;
    #400     car = 1'b0;
    #250     car = 1'b1;
    #350     car = 1'b0;
    #200     $stop;
end

```

Fig. 2-3 testbench 的程式片段

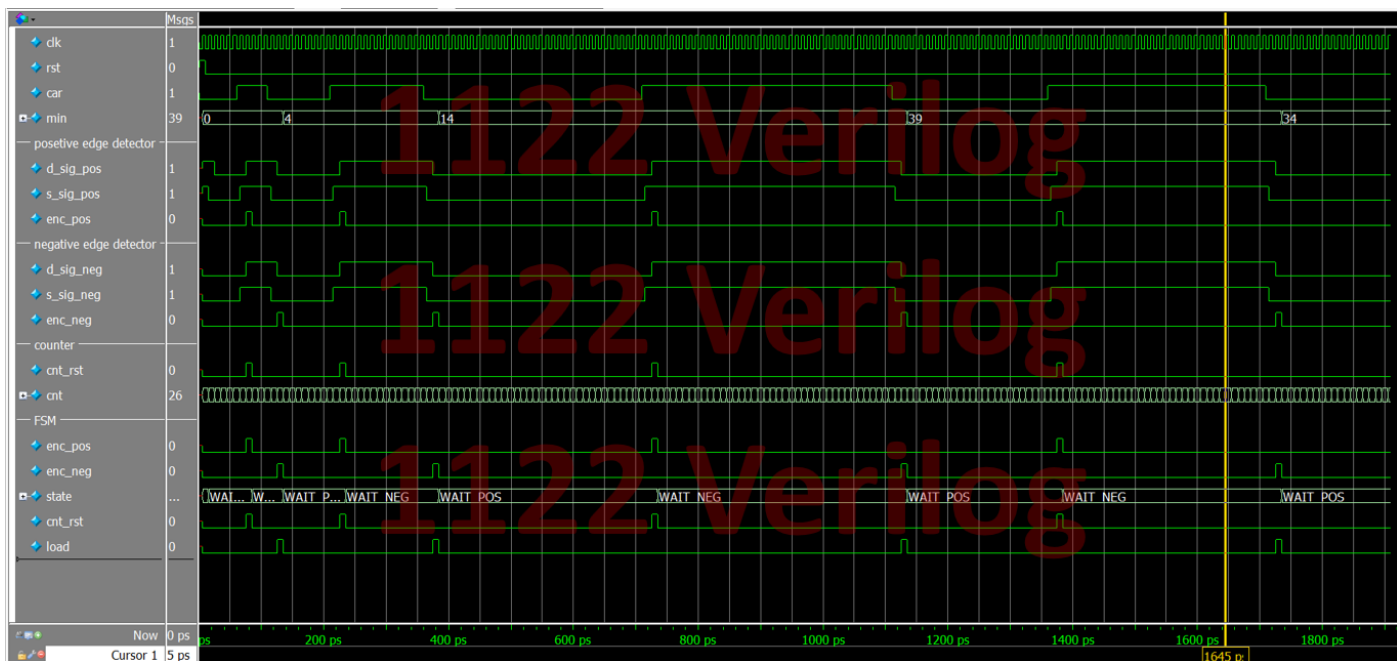


Fig. 2-4 波型圖

parking_space.v

主程式碼截圖：

//放主程式碼截圖

```

1  module parking_space(
2      input clk,
3      input rst,
4      input car,
5      output reg [15:0] min
6  );
7
8      reg [15:0] cnt;
9      reg enc_pos, enc_neg;
10     reg cnt_rst;
11     reg [1:0] ps, ns;
12     reg [(8*8-1):0] state;
13     reg load;
14     reg s_sig_pos, d_sig_pos;
15     reg s_sig_neg, d_sig_neg;
16
17     parameter IDLE      = 2'd0,
18                WAIT_POS  = 2'd1,
19                WAIT_NEG  = 2'd2;
20
21     always @(*)
22     begin
23         case (ps)
24             2'd0: state = "IDLE";
25             2'd1: state = "WAIT_POS";
26             2'd2: state = "WAIT_NEG";
27         endcase
28     end
29
30
31     //positive edge detector
32     always @(posedge clk)
33     begin
34         if (rst)
35         begin
36             s_sig_pos  <= 1'b1;
37             d_sig_pos  <= 1'b1;
38             enc_pos    <= 1'b1;
39         end
40         else
41         begin
42             {d_sig_pos, s_sig_pos} <= {s_sig_pos, car};
43             enc_pos <= s_sig_pos & ~d_sig_pos;
44         end

```

```

45     end
46
47     //negative edge detector
48     always @(posedge clk)
49     begin
50         if (rst)
51         begin
52             s_sig_neg    <= 1'b0;
53             d_sig_neg    <= 1'b0;
54             enc_neg      <= 1'b0;
55         end
56         else
57         begin
58             {d_sig_neg, s_sig_neg} <= {s_sig_neg, car};
59             enc_neg <= d_sig_neg & ~s_sig_neg;
60         end
61     end
62
63     always @(posedge clk)
64     begin
65         if (rst || cnt_rst)
66             cnt <= 16'd0;
67         else
68             cnt <= cnt + 1'b1;
69     end
70
71     always @(posedge clk)
72     begin
73         if (rst)
74             ps <= IDLE;
75         else
76             ps <= ns;
77     end
78
79     always @(*)
80     begin
81         case (ps)
82             IDLE:    ns = WAIT_POS;
83
84             WAIT_POS:
85             begin
86                 cnt_rst = 1'b0;
87                 load    = 1'b0;
88                 if (enc_pos)

```

```

89      begin
90          ns = WAIT_NEG;
91          cnt_rst = 1'b1;
92      end
93      else
94          ns = WAIT_POS;
95      end
96  WAIT_NEG:
97      begin
98          cnt_rst = 1'b0;
99          if (enc_neg)
100              begin
101                  ns = WAIT_POS;
102                  load = 1'b1;
103              end
104              else
105                  ns = WAIT_NEG;
106              end
107      endcase
108  end
109
110  always @(posedge clk)
111      begin
112          if(rst)
113              min <= 16'd0;
114          else if (load)
115              min <= cnt;
116          else
117              min <= min;
118      end
119  endmodule

```

parking_space_test.v

測試程式碼截圖：

//放測試程式碼截圖

附錄:

```
//// posetive edge detector
always @(posedge clk) begin
    if(rst) begin
        s_sig_pos    <= 1'b1;
        d_sig_pos    <= 1'b1;
        enc_pos      <= 1'b0;
    end
    else begin
        {d_sig_pos, s_sig_pos} <= {s_sig_pos, car};
        enc_pos      <= s_sig_pos & ~d_sig_pos;
    end
end
end
```

Fig. 3-1 posetive edge detector 程式碼

```
//// decleared
reg      [(8*8-1):0] state; // 8 bits for one word, total 8 words (state word)
```

```
case(ps)
    IDLE: begin
        state      = "IDLE";
        ns         = WAIT_POS;
    end
```

Fig. 3-2 FSM 的狀態文字顯示(需轉成 ASCII 碼)